

Data Mining

Lab - 14

¥@\$# Kakadiya | 23010101121 |
17/9/2025

Implement K- Means without Library

Sample data points

```
data = [ [1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13], [50, 51], [51, 52], [52, 53] ]
```

```
In [11]: import math # Import the math library for mathematical operations
```

```
In [12]: # Sample data points for clustering
data = [
    [1, 2], [2, 3], [3, 4],
    [10, 11], [11, 12], [12, 13],
    [50, 51], [51, 52], [52, 53]
]
```

```
In [13]: # Function to calculate the Euclidean distance between two points
def distance(x1,x2):
    return math.sqrt(((x1[0] - x2[0])**2) + ((x1[1] - x2[1])**2))
```

```
In [14]: distance([1,1],[1,1])
```

```
Out[14]: 0.0
```

```
In [15]: # Function to update the cluster center by calculating the mean of all points in the cluster
def update_cluster_center(cluster_data):
    sum = [0,0] # Initialize sum for x and y coordinates
    for i in cluster_data:
        sum[0] = sum[0] + i[0] # Sum x coordinates
        sum[1] = sum[1] + i[1] # Sum y coordinates
    return [sum[0]/len(cluster_data),sum[1]/len(cluster_data)] # Return the new cluster center
```

```
In [16]: update_cluster_center([[1,1],[2,2],[1,1]])
```

```
Out[16]: [1.3333333333333333, 1.3333333333333333]
```

Now Implement code

```
In [17]: import numpy as np

def kmeans_du(k,data):
    # select random center
    center_data = [data[np.random.randint(0,len(data))]] for i in range(0,k)]
    print(center_data)

    #cluster data
    cluster_data = [[] for i in range(0,k)]
    for i in range(0,k):
        cluster_data[i].append(center_data[i])
    print(cluster_data)

    for j in range(0,5):
        cluster_data = [[] for i in range(0,k)]
        for d in data:
            mindistance = []
            for i in range(0,k):
                mindistance.append(distance(center_data[i],d))
            print(d ,"-->",mindistance)
            cluster_data[mindistance.index(min(mindistance))].append(d)

        # print Cluster data

        for i in range(0,k):
            print(i,"-->",cluster_data[i])

        # update Cluster center
        for i in range(0,k):
            if cluster_data[i]: # Check if the cluster is not empty
                center_data[i] = update_cluster_center(cluster_data[i])
        print("NEW Cluster Center",center_data)
```

```
In [18]: kmeans_du(3,data)
```

```
[[[51, 52], [11, 12], [52, 53]]
[[[51, 52]], [[11, 12]], [[52, 53]]]
[1, 2] --> [70.71067811865476, 14.142135623730951, 72.12489168102785]
[2, 3] --> [69.29646455628166, 12.727922061357855, 70.71067811865476]
[3, 4] --> [67.88225099390856, 11.313708498984761, 69.29646455628166]
[10, 11] --> [57.982756057296896, 1.4142135623730951, 59.39696961966999]
[11, 12] --> [56.568542494923804, 0.0, 57.982756057296896]
[12, 13] --> [55.154328932550705, 1.4142135623730951, 56.568542494923804]
[50, 51] --> [1.4142135623730951, 55.154328932550705, 2.8284271247461903]
[51, 52] --> [0.0, 56.568542494923804, 1.4142135623730951]
[52, 53] --> [1.4142135623730951, 57.982756057296896, 0.0]
0 --> [[50, 51], [51, 52]]
1 --> [[1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13]]
2 --> [[52, 53]]
NEW Cluster Center [[50.5, 51.5], [6.5, 7.5], [52.0, 53.0]]
[1, 2] --> [70.0035713374682, 7.7781745930520225, 72.12489168102785]
[2, 3] --> [68.58935777509511, 6.363961030678928, 70.71067811865476]
[3, 4] --> [67.17514421272202, 4.949747468305833, 69.29646455628166]
[10, 11] --> [57.27564927611035, 4.949747468305833, 59.39696961966999]
[11, 12] --> [55.86143571373726, 6.363961030678928, 57.982756057296896]
[12, 13] --> [54.44722215136416, 7.7781745930520225, 56.568542494923804]
[50, 51] --> [0.7071067811865476, 61.518289963229634, 2.8284271247461903]
[51, 52] --> [0.7071067811865476, 62.932503525602726, 1.4142135623730951]
[52, 53] --> [2.1213203435596424, 64.34671708797582, 0.0]
0 --> [[50, 51], [51, 52]]
1 --> [[1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13]]
2 --> [[52, 53]]
NEW Cluster Center [[50.5, 51.5], [6.5, 7.5], [52.0, 53.0]]
[1, 2] --> [70.0035713374682, 7.7781745930520225, 72.12489168102785]
[2, 3] --> [68.58935777509511, 6.363961030678928, 70.71067811865476]
[3, 4] --> [67.17514421272202, 4.949747468305833, 69.29646455628166]
[10, 11] --> [57.27564927611035, 4.949747468305833, 59.39696961966999]
[11, 12] --> [55.86143571373726, 6.363961030678928, 57.982756057296896]
[12, 13] --> [54.44722215136416, 7.7781745930520225, 56.568542494923804]
[50, 51] --> [0.7071067811865476, 61.518289963229634, 2.8284271247461903]
[51, 52] --> [0.7071067811865476, 62.932503525602726, 1.4142135623730951]
[52, 53] --> [2.1213203435596424, 64.34671708797582, 0.0]
0 --> [[50, 51], [51, 52]]
1 --> [[1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13]]
2 --> [[52, 53]]
NEW Cluster Center [[50.5, 51.5], [6.5, 7.5], [52.0, 53.0]]
[1, 2] --> [70.0035713374682, 7.7781745930520225, 72.12489168102785]
[2, 3] --> [68.58935777509511, 6.363961030678928, 70.71067811865476]
[3, 4] --> [67.17514421272202, 4.949747468305833, 69.29646455628166]
[10, 11] --> [57.27564927611035, 4.949747468305833, 59.39696961966999]
[11, 12] --> [55.86143571373726, 6.363961030678928, 57.982756057296896]
[12, 13] --> [54.44722215136416, 7.7781745930520225, 56.568542494923804]
[50, 51] --> [0.7071067811865476, 61.518289963229634, 2.8284271247461903]
[51, 52] --> [0.7071067811865476, 62.932503525602726, 1.4142135623730951]
[52, 53] --> [2.1213203435596424, 64.34671708797582, 0.0]
0 --> [[50, 51], [51, 52]]
1 --> [[1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13]]
2 --> [[52, 53]]
NEW Cluster Center [[50.5, 51.5], [6.5, 7.5], [52.0, 53.0]]
[1, 2] --> [70.0035713374682, 7.7781745930520225, 72.12489168102785]
[2, 3] --> [68.58935777509511, 6.363961030678928, 70.71067811865476]
[3, 4] --> [67.17514421272202, 4.949747468305833, 69.29646455628166]
[10, 11] --> [57.27564927611035, 4.949747468305833, 59.39696961966999]
[11, 12] --> [55.86143571373726, 6.363961030678928, 57.982756057296896]
[12, 13] --> [54.44722215136416, 7.7781745930520225, 56.568542494923804]
[50, 51] --> [0.7071067811865476, 61.518289963229634, 2.8284271247461903]
[51, 52] --> [0.7071067811865476, 62.932503525602726, 1.4142135623730951]
[52, 53] --> [2.1213203435596424, 64.34671708797582, 0.0]
0 --> [[50, 51], [51, 52]]
1 --> [[1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13]]
```

```
2 --> [[52, 53]]  
NEW Cluster Center [[50.5, 51.5], [6.5, 7.5], [52.0, 53.0]]
```

Implement K-Medoids without Library

Sample data points

```
data = [ [1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13], [50, 51], [51, 52], [52, 53] ]
```

```
In [19]: import random # Import the random library for random sampling  
import math # Import the math library for mathematical operations
```

```
In [20]: # Function to calculate the Euclidean distance between two points  
def euclidean_distance(p1, p2):  
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(p1, p2)))
```

```
In [21]: # Function to assign each data point to the nearest medoid  
def assign_points(data, medoids):  
    clusters = {i: [] for i in range(len(medoids))} # Initialize empty clusters  
    for point in data:  
        distances = [euclidean_distance(point, medoid) for medoid in medoids] # Calculate distances  
        nearest = distances.index(min(distances)) # Find the index of the nearest medoid  
        clusters[nearest].append(point) # Assign the point to the nearest cluster  
    return clusters # Return the clusters
```

```
In [22]: def calculate_cost(clusters, medoids):  
    cost = 0  
    for i, points in clusters.items():  
        for p in points:  
            cost += euclidean_distance(p, medoids[i])  
    return cost
```

```
In [23]: def calculate_cost(clusters, medoids):  
    # Calculate the total cost (sum of distances) of the current clustering  
    cost = 0  
    for i, points in clusters.items():  
        for p in points:  
            cost += euclidean_distance(p, medoids[i])  
    return cost  
  
def k_medoids(data, k, max_iter=100):  
    # Step 1: Randomly select initial medoids from the data points  
    medoids = random.sample(data, k)  
  
    # Iterate for a maximum number of times or until convergence  
    for _ in range(max_iter):  
        # Assign each data point to the nearest medoid to form clusters  
        clusters = assign_points(data, medoids)  
        # Calculate the current cost of the clustering  
        current_cost = calculate_cost(clusters, medoids)  
  
        # Keep track of the best medoids found so far  
        best_medoids = medoids[:]  
        improved = False # Flag to check if the clustering improved in this iteration  
  
        # Step 2: Try swapping each medoid with a non-medoid data point  
        for i in range(len(medoids)):  
            for candidate in data:  
                # Check if the candidate data point is not already a medoid  
                if candidate not in medoids:
```

```

        # Create a new set of medoids with the swap
        new_medoids = medoids[:]
        new_medoids[i] = candidate
        # Assign points to the new medoids
        new_clusters = assign_points(data, new_medoids)
        # Calculate the cost of the new clustering
        new_cost = calculate_cost(new_clusters, new_medoids)

        # If the new clustering has a lower cost, update the best medoids and current cost
        if new_cost < current_cost:
            best_medoids = new_medoids
            current_cost = new_cost
            improved = True # Set improved flag to True

    # Update the medoids to the best set found in this iteration
    medoids = best_medoids
    # If no improvement was made in this iteration, the algorithm has converged
    if not improved:
        break # convergence

# After the iterations, assign points one last time to the final medoids
final_clusters = assign_points(data, medoids)
# Return the final medoids and the corresponding clusters
return medoids, final_clusters

```

```

In [24]: # Set the number of clusters
k = 3
# Run the K-Medoids algorithm on the data
medoids, clusters = k_medoids(data, k)

```

```

In [25]: # Print the final medoids
print("Final Medoids:", medoids)
# Print the points in each final cluster
print("Clusters:")
for i, points in clusters.items():
    print(f"Cluster {i+1}: {points}")

```

```

Final Medoids: [[51, 52], [2, 3], [11, 12]]
Clusters:
Cluster 1: [[50, 51], [51, 52], [52, 53]]
Cluster 2: [[1, 2], [2, 3], [3, 4]]
Cluster 3: [[10, 11], [11, 12], [12, 13]]

```